

## Documento de Arquitetura: Esteira DevSecOps (Shift Left)

Este projeto adota o paradigma *Shift Left*, antecipando de forma proativa os processos de segurança para as fases mais precoces do ciclo de desenvolvimento, reduzindo os custos e o esforço na correção de vulnerabilidades.

### 1. Orquestração de CI/CD

- **Ferramenta:** GitHub Actions.
- **Justificativa:** Oferece integração nativa e fluida com o repositório de código, permitindo a construção de *pipelines* declarativas em YAML. É altamente compatível com o ecossistema de segurança e facilita a configuração de *Quality Gates*, interrompendo o fluxo imediatamente (*Build Breaker*) se um desenvolvedor cometer um erro grave.

### 2. Containerização (Docker)

- **Estratégia:** Utilização de imagens Docker com *multi-stage build*.
- **Justificativa:** Esta prática garante que ferramentas de compilação e dependências de desenvolvimento (por exemplo, pacotes não essenciais num ambiente Python/Django) não sejam incluídas na imagem final, minimizando drasticamente a superfície de ataque. A imagem será armazenada num *Registry* privado (como o GitHub Container Registry) com controle de acessos restrito.

### 3. Infraestrutura como Código (IaC)

- **Ferramenta:** Terraform.
- **Justificativa:** Permite provisionar e gerir toda a infraestrutura em nuvem de forma automatizada e versionada. Ao tratar a infraestrutura como código, torna-se possível aplicar testes de segurança automatizados antes que qualquer recurso seja efetivamente criado no ambiente de produção.

### 4. Observabilidade e Monitoramento

- **Ferramentas:** Prometheus (para métricas) e Grafana (para visualização de *dashboards*).
- **Justificativa:** Essas ferramentas garantem o monitoramento contínuo através da coleta estruturada e análise contínua de métricas de servidores e aplicações para

identificar comportamentos anômalos. Em caso de incidentes, alertas automatizados são acionados (via Slack ou email), permitindo o isolamento rápido de componentes comprometidos da rede.

## 5. Integração das Ferramentas de Segurança e Quality Gates

A esteira está desenhada para atuar com bloqueios rigorosos nas seguintes fases:

- **Pré-commit / Push Inicial (GitGuardian):** Atua na prevenção de vazamento de credenciais. *Bloqueio:* Se uma chave de API, token ou senha de base de dados for detetada no momento do *push*, a pipeline bloqueia o envio do código imediatamente.
- **Fase de Build / Integração (SonarQube - SAST):** Realiza a análise estática ("Caixa Branca") do código fonte para identificar falhas estruturais, como *SQL Injection* ou *Cross-Site Scripting* (XSS). *Bloqueio:* A esteira falha se o código não atingir a nota mínima de cobertura de segurança e qualidade estabelecida pelo *Quality Gate*.
- **Pós-Build / Antes do Deploy (Trivy e Checkov):** O Trivy faz a varredura de vulnerabilidades na imagem Docker recém-criada, enquanto o Checkov analisa os ficheiros do Terraform. *Bloqueio:* O *deploy* é cancelado se o Trivy detectar vulnerabilidades críticas de sistema operativo nos pacotes, ou se o Checkov identificar configurações inseguras na infraestrutura (como portas abertas indevidamente).
- **Ambiente de Staging (OWASP ZAP - DAST):** Executa testes dinâmicos de segurança ("Caixa Preta") interagindo com a aplicação em execução para detetar falhas de autenticação ou de gestão de sessões. *Bloqueio:* O *deploy* final para Produção é abortado se forem validadas vulnerabilidades dinâmicas críticas.